

# CYBERSECURITY INTERNSHIP TRAINING REPORT

## Cybersecurity Fundamentals



|                                                                                     |              |                                         |
|-------------------------------------------------------------------------------------|--------------|-----------------------------------------|
|  | Submitted By | : ABHI DHRUVE                           |
|  | Course       | : BCA                                   |
|  | Internship   | : ApexPlanet Software Pvt. Ltd.         |
|  | Domain       | : CYBER SECURITY AND<br>ETHICAL HACKING |
|  | Year         | : 2026                                  |
|  | Date         | : 31 AUGUST 2026                        |



# CYBERSECURITY INTERNSHIP INDEX

Hands-on modules to build practical cybersecurity skills.

**1**

## RECONNAISSANCE

01



- Passive Recon : Whois, Nslookup, Google Dorking, Shodan.
- Active Recon : Ping Sweep, Banner Grabbing.

**2**

## PORT & SERVICE SCANNING

02



- Nmap TCP & UDP Scans (-sS, -sU)
- Service version Detection (-sV)
- OS Detection (-O).
- Create a scan report for target systems.

**3**

## VULNERABILITY SCANNING

03



- Setup OpenVAS or Nessus Essentials.
- Scan target VM.
- Analyze Vulnerability reports (critical, High, medium, low).

**4**

## PACKET ANALYSIS WITH WIRESHARK

04



- Capture HTTP, FTP & DNS traffic in Wireshark.
- Filter credentials from unencrypted FTP traffic.
- Analyze SYN flood attack (simulate with hping3).

**5**

## FIREWALL BASICS

05



- Create simple iptables rules (allow/deny specific ports).
- Demonstrate blocking a port scan attempt.



### LEARNING PATH

Follow this path to become a cybersecurity professional.



Learn  
(Concepts)



Practice  
(Labs)



Analyze  
(Tools)



Secure  
(Mindset)



# 1 Reconnaissance

- **Passive Recon** : Whois Nslookup, Google Dorking Shodan
- **Active Recon** : Ping Sweep Banner Grabbing

## 1 Reconnaissance :

Reconnaissance is the first phase of cyber security and ethical hacking. It involves collecting information about a target before performing security testing. The purpose is to understand the target domain, IP addresses, technologies and publicly available information.

## **2 Type of Reconnaissance:**

- **Passive** = collecting information without directly interacting with target system.
- **Active** = directly interacting with the target such as scanning ports or service.

### **1 Passive :**

Passive Reconnaissance collects information from publicly available sources, the target system is generally not directly contacted.

#### **1 WHOIS**

WHOIS is a service used to find information about a domain name or IP address.

- It may provide
  - Domain registration details
  - Domain register information
  - Registration and expiry dates
  - Name servers
  - Technical information

Example

whois example.com

## 2 NSLOOKUP

NSLOOKUP (Name Server Lookup) is a command line tool to query the domain name system.

It can help identify

- IP address of a domain
- DNS servers
- Mail servers
- Other DNS Records

Example

nslookup example.com

## 3 Google Dorking

Google Dorking also called Google hacking uses advanced search operators to locate publicly indexed information

## **Operator   Purpose**

site:            Search within a specific website/domain

filetype:       Search for a specific file type

intitle:         Search words in page titles

inurl:           Search words in URLs

## 4 Shodan

Shodan is search engine designed to discover information about internet connected devices and services

It can provide information such as

- Public IP addresses
- Service banners
- Open services
- Device and software information
- Organization and network information

## 2 Port & service Scanning:

- Nmap TCP & UDP Scans (-sS -sU)
- Service version Detection (-sV)
- OS Detection (-O).
- Create a scan report for target systems

### 1 Nmap TCP & UDP Scans (-sS -sU)

Nmap is a network scanning tool used to discover hosts, open ports, and services on a network

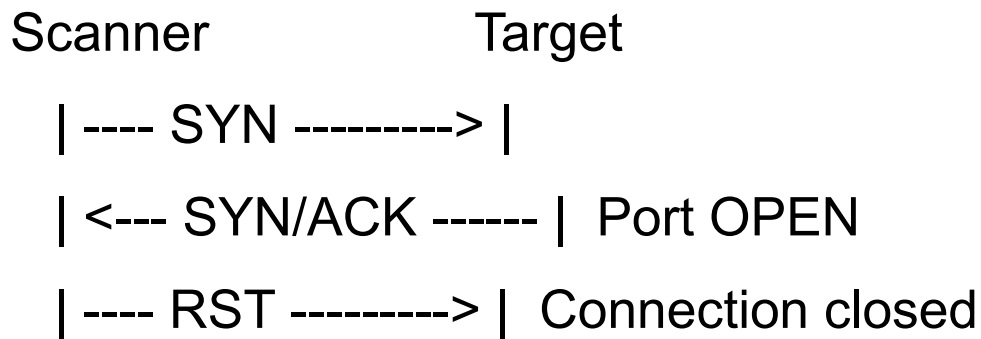
### 1 TCP SYN Scan (-sS)

Command

```
nmap -sS <target>
```

TCP SYN scan is also called a half open scan. nmap sends a SYN packet to a target port.

## Working



- SYN/ACK received = port is open
- RST received = port is closed
- No response = port may be filtered

## Advantage

- Fast scanning method
- Does not complete the full TCP connection
- Useful for finding open TCP ports
- Commonly used to network security testing

## Example

`nmap -sS 192.168.1.1`



## 2 UDP scan (-sU)

### Command

`nmap -sU <target>`

UDP scan check for services running on UDP ports

### Working

| Scanner             | Target          |
|---------------------|-----------------|
| ---- UDP Packet --> |                 |
|                     |                 |
| <--- No Response -- | Open / Filtered |

OR

| <--- ICMP Error --- | Port CLOSED

- UDP response received = port is open
- ICMP port Unreachable = port is closed
- No response = Port is Open or Filtered

## Common UDP services

| Port  | Service |
|-------|---------|
| 53    | DNS     |
| 67/68 | DHCP    |
| 69    | TFTP    |
| 123   | NTP     |
| 161   | SNMP    |

## Example

```
nmap -sU 192.168.1.1
```

## 2 Service version Detection

Service version Detection is an Nmap technique used to identify the service running on open port and its version number

Command

```
nmap -sV <target>
```

Example

```
nmap -sV 192.168.1.10
```

Example output

| PORT     | STATE | SERVICE | VERSION          |
|----------|-------|---------|------------------|
| 22/tcp   | open  | ssh     | OpenSSH 8.2      |
| 80/tcp   | open  | http    | Apache httpd 2.4 |
| 3306/tcp | open  | mysql   | MySQL 8.0        |

## Explanation

| Port | Service | Version     |
|------|---------|-------------|
| 22   | SSH     | OpenSSH 8.2 |
| 80   | HTTP    | Apache 2.4  |
| 3306 | MySQL   | MySQL 8.0   |

## Advantage

- Identifies services running on open ports
- Detects software and version information
- Help administrators identify outdated software
- Useful for authorized security assessments

## Impartment options

```
nmap -sV --version-intensity 5 <target>
```

### 3 OS Detection (-O)

OS Detection is an Nmap feature used to identify the operating system running on a target device such as Linux Windows macOS or network devices

#### Command

```
nmap -O <target>
```

#### Example

```
nmap -O 192.168.1.10
```

how its works

Nmap sends different types of TCP UDP and ICMP packets to the target and analyzes the resource.

Every operating system has slightly different network behavior called a TCP/IP fingerprint

Nmap Scanner



| Sends network probes



Target System



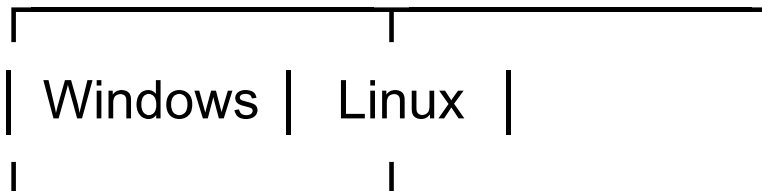
| Sends responses



TCP/IP Fingerprinting



Possible Operating System





## Example output

Nmap scan report for 192.168.1.10

OS details: Linux 5.x

Network Distance: 1 hop

Information detected

Nmap may identify

- Operating system name
- OS version or family
- Device type
- Network distance
- TCP/IP characteries

## Impartment options

| Command                                            | Purpose                                                 |
|----------------------------------------------------|---------------------------------------------------------|
| <code>nmap -O &lt;target&gt;</code>                | Perform OS detection                                    |
| <code>nmap -A &lt;target&gt;</code>                | OS detection + version detection + scripts + traceroute |
| <code>nmap -O --osscan-guess &lt;target&gt;</code> | Make a more aggressive OS guess                         |

## Advantage

- Help Identify devices on a network
- Useful for network inventory
- Helps security terms understand the environment
- Can identify potentially outdated operating systems
- Useful during authorized penetration testing and security assessments

## 3 vulnerability Scanning:

- Setup OpenVAS or Nessus Essentials
  - Scan target VM
  - Analyze Vulnerability reports  
(critical,High,medium,low)
- 
- Setup OpenVAS or Nessus Essentials

### Nessus Essentials

- 1.Download Nessus from the official Tenable site
- 2.Install it on windows or Linux
- 3.Start the Nessus service
- 4.Open your browser and go to <http://localhost:8834>
- 5.Choose Nessus Essentials
- 6.Register for an activation code using Tenable registration process
- 7.Create your administrator account

8. Wait for plugins to download and initialize
9. Create a new scan
10. New scan > Basic Network Scan
11. Enter the IP address of own lab machine VM as the target
12. Start the scan
13. Review vulnerabilities by severity
14. Critical > High > Medium > Low > Informational

- Scan test VM (Metasploitable 2)

### 1. Find the Metasploitable 2 IP

On Metasploitable 2:

ifconfig

Look for something like:

inet addr: 192.168.56.101

Your IP will probably be different.

From your scanning machine, verify connectivity:

ping 192.168.56.101

## 2. Nessus Essentials scan

Open:

<https://localhost:8834>

Then:

**New Scan → Basic Network Scan**

Enter:

Name: Metasploitable 2 Lab Scan

Targets: 192.168.56.101

Click **Save** → **Launch**.

Wait for the scan to finish.

## 3. Analyze the results

Look at:

- **Critical** vulnerabilities
- **High** vulnerabilities
- **Medium** vulnerabilities
- Open ports/services
- CVE identifiers
- Vulnerable software versions
- Recommended remediation

For example, you may see services such as:

21/tcp FTP  
22/tcp SSH  
23/tcp Telnet  
80/tcp HTTP  
139/tcp SMB  
445/tcp SMB  
3306/tcp MySQL

The exact results depend on your VM/network configuration.

- Analyze Vulnerability Reports  
(Critical/High/Medium/Low)

After scanning your **Metasploitable 2 test VM** with Nessus Essentials or OpenVAS, vulnerabilities are normally grouped by severity. The purpose of analysis is to understand **what is vulnerable, how serious it is, and what should be fixed first.**

## 1. Critical

**Critical** vulnerabilities have the highest priority. They can potentially allow an attacker to gain **remote control, execute arbitrary code, bypass security controls, or cause major compromise.**

**What to check:**

- CVE number
- Vulnerable service/software
- Affected port
- Exploitation status
- Scanner evidence
- Recommended fix

**Action:** Fix immediately or isolate the vulnerable service.

## 2. High

**High** vulnerabilities represent a significant security risk. They may allow unauthorized access, privilege escalation, sensitive-data exposure, or other serious attacks.

## **Example:**

Port: 445

Service: SMB

Severity: High

Issue: Vulnerable SMB implementation

**Action:** Prioritize patching, upgrading, disabling, or restricting the affected service.

## **3. Medium**

**Medium** vulnerabilities generally require additional conditions or access to be successfully exploited, but they should still be addressed.

Examples include:

- Information disclosure
- Weak configurations
- Outdated software
- Missing security controls
- Services unnecessarily exposed



**Action:** Schedule remediation and improve the configuration.

#### **4. Low**

**Low** vulnerabilities have relatively limited impact.

Examples:

- Minor information disclosure
- Security-hardening recommendations
- Weak but non-critical configurations

**Action:** Fix during normal maintenance and use them to improve overall security posture.

---

#### **5. Informational**

Some scanners also report **Informational** findings. These aren't necessarily vulnerabilities.

For example:

Port 80 is open

Web server detected

Operating system detected

Service version detected

They provide useful information for understanding the attack surface.

### **What to record for each vulnerability**

For your lab report, record:

Vulnerability:

Severity:

CVE:

Affected Host:

Port:

Service:

Description:

Evidence:

Potential Impact:

Recommended Remediation:

Status:

## 4 packet analysis with wireshark:

- Capture HTTP FTP & DNS traffic in Wireshark
- Filter credentials from unencrypted FTP traffic
- Analyze SYN flood attack (simulate with hping3).

- Capture HTTP FTP & DNS traffic in Wireshark

### 1. Start Wireshark

Open **Wireshark** on your Kali/Linux machine.

1. Find the network interface connected to Metasploitable 2.

2. Select it, for example:

- eth0
- ens33
- enp0s3

3. Double-click the interface to start capturing.

Make sure Kali and Metasploitable 2 are connected to the same **Host-Only/NAT lab network**.

---

## **2. Capture HTTP Traffic**

First, find your Metasploitable 2 IP:

ifconfig

Suppose it is:

192.168.56.101

From Kali, open:

http://192.168.56.101

Now return to Wireshark and enter this **Display Filter**:

http

Press **Enter**.

You should see packets such as:

GET / HTTP/1.1

HTTP/1.1 200 OK

**Analyze HTTP**

Click an HTTP packet and expand:

## **Hypertext Transfer Protocol**

Look for:

- Request Method
- Host
- Request URI
- Response Code
- User-Agent
- Source IP
- Destination IP
- TCP port 80

You can also right-click the packet:

**Follow → TCP Stream**

---

### **3. Capture FTP Traffic**

From Kali, connect to your Metasploitable 2 FTP server:

```
ftp 192.168.56.101
```

After generating FTP traffic, return to Wireshark.

Use:

ftp

or:

tcp.port == 21

You should see packets such as:

220 FTP server ready

USER ...

331 Password required

PASS ...

230 Login successful

## **Analyze FTP**

Select an FTP packet and expand:

### **File Transfer Protocol**

Check:

- FTP command
- Server response
- Source IP
- Destination IP
- TCP port 21

- **Lab note:** FTP is plaintext, so credentials can potentially appear in the capture. Use only your Metasploitable 2 lab account.
- 

#### **4. Capture DNS Traffic**

Generate a DNS request from Kali:

nslookup example.com

or:

dig example.com

In Wireshark, use:

dns

Press **Enter**.

You should see DNS packets containing:

Standard query

Standard query response

Click a DNS packet and expand:

## Domain Name System

Look for:

- Query name
- Query type
- Response
- Answer
- Source IP
- Destination IP
- UDP port 53

### 5. Useful Wireshark Filters

| Purpose   | Wireshark filter |
|-----------|------------------|
| HTTP      | http             |
| FTP       | ftp              |
| FTP port  | tcp.port == 21   |
| DNS       | dns              |
| DNS port  | udp.port == 53   |
| HTTP port | tcp.port == 80   |



## Purpose

## Wireshark filter

Metasploitable traffic `ip.addr == 192.168.56.101`

HTTP + FTP + DNS `http or ftp or dns`

## 6. Save the Capture

After capturing the traffic:

**File → Save As**

Save it as:

`metasploitable_traffic.pcapng`

You can then reopen the file later for analysis.

## **What your practical should demonstrate**

Kali Linux

|

| HTTP → TCP 80

| FTP → TCP 21

| DNS → UDP 53



Metasploitable 2

- Filter credentials from unencrypted FTP traffic.

For your **Metasploitable 2 lab**, FTP credentials can be visible because traditional FTP sends authentication data without encryption.

### 1. Start Wireshark

Select the interface connected to your Metasploitable 2 VM and start capturing.

Then generate FTP traffic from your **lab machine**:

```
ftp <METASPLOITABLE-IP>
```

Log in with your lab/test account.

## 2. Filter FTP traffic

In Wireshark's **Display Filter** bar, enter:

ftp

Press **Enter**.

You should see FTP commands such as:

USER <username>

PASS <password>

## 3. Filter specifically for authentication commands

Use:

```
ftp.request.command == "USER" ||
```

```
ftp.request.command == "PASS"
```

This focuses the packet list on FTP authentication commands.

You may see:

USER msfadmin

PASS <password>

**Do not use or share real credentials.** For your lab report, redact the password, e.g.:

USER labuser

PASS [REDACTED]

#### **4. Follow the FTP session**

Right-click an FTP packet:

**Follow → TCP Stream**

This lets you view the complete FTP conversation. Because ordinary FTP is unencrypted, authentication commands may appear as readable text.

#### **5. Save evidence**

Take a screenshot showing:

- Wireshark filter
- USER command
- PASS command
- Source/destination IP
- TCP port 21

- Analyze SYN flood attack (simulate with hping3).

You can practice this safely against **your own Metasploitable 2 VM** in an isolated network. A SYN flood attempts to create many half-open TCP connections, so the goal here is to **observe the traffic in Wireshark**, not disrupt any real system.

### 1. Find Metasploitable 2 IP

On Metasploitable 2:

ifconfig

Example:

192.168.56.101

Verify connectivity from Kali:

ping 192.168.56.101

## 2. Start Wireshark

Select the interface connected to your lab network and start capturing.

Use this display filter:

```
tcp.flags.syn == 1 && tcp.flags.ack == 0
```

This displays TCP **SYN packets without ACK**.

## 3. Generate a controlled SYN test

For a safe lab demonstration, use a **very small, rate-limited test** against your Metasploitable VM:

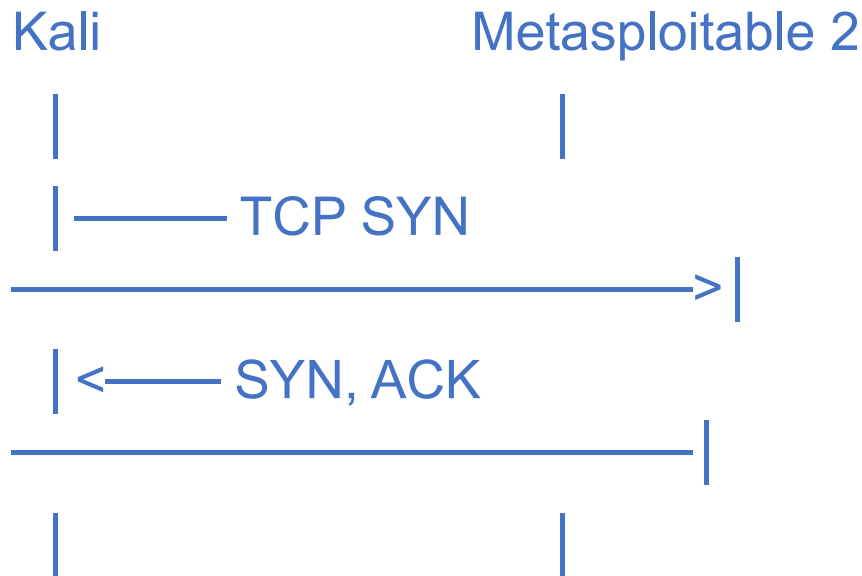
```
sudo hping3 -S -p 80 -c 20 -i u100000  
192.168.56.101
```

Where:

- -S → set SYN flag
- -p 80 → target TCP port 80
- -c 20 → send only 20 packets
- -i u100000 → 100,000  $\mu$ s (0.1 second) between packets
- 192.168.56.101 → your lab VM

## 4. Analyze packets in Wireshark

You should see:



For an actual SYN-flood pattern, look for:

- Large numbers of SYN packets
- Many connections targeting the same destination port
- SYN packets without corresponding completed handshakes
- Increasing half-open connections
- Repeated source addresses, if present

## **5. Useful Wireshark filters**

### **SYN packets:**

`tcp.flags.syn == 1`

### **SYN without ACK:**

`tcp.flags.syn == 1 && tcp.flags.ack == 0`

### **Traffic to HTTP port 80:**

`tcp.dstport == 80`

### **Combine both:**

`tcp.flags.syn == 1 && tcp.flags.ack == 0 &&  
tcp.dstport == 80`



## 5 firewall basics:

- Create simple iptables rules (allow/deny specific ports).
- Demonstrate blocking a port scan attempt.
- Create simple iptables rules (allow/deny specific ports).

For your **Kali ↔ Metasploitable 2 cybersecurity lab**, iptables can control which network ports are allowed or blocked.

Run these commands only on a VM/system you own or are authorized to administer. Be careful when changing firewall rules over SSH—you can lock yourself out.

### **1. Check existing rules**

```
sudo iptables -L -n -v
```

---

### **2. Allow SSH — Port 22**

```
sudo iptables -A INPUT -p tcp --dport 22 -j  
ACCEPT
```

This allows incoming TCP connections to **port 22**.

---

### 3. Allow HTTP — Port 80

```
sudo iptables -A INPUT -p tcp --dport 80 -j  
ACCEPT
```

This allows incoming HTTP traffic.

---

### 4. Allow HTTPS — Port 443

```
sudo iptables -A INPUT -p tcp --dport 443 -j  
ACCEPT
```

### 5. Deny FTP — Port 21

```
sudo iptables -A INPUT -p tcp --dport 21 -j  
DROP
```

This silently drops incoming FTP connections.

Alternatively:

```
sudo iptables -A INPUT -p tcp --dport 21 -j  
REJECT
```

DROP silently discards the packet, while  
REJECT sends a rejection response.

---

## **6. Deny Telnet — Port 23**

```
sudo iptables -A INPUT -p tcp --dport 23 -j  
DROP
```

---

## **7. Allow only specific ports**

For example, allow SSH and HTTP:

```
sudo iptables -A INPUT -p tcp --dport 22 -j  
ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j  
ACCEPT
```

Then deny other incoming TCP traffic:

```
sudo iptables -A INPUT -p tcp -j DROP
```

**Rule order matters.** iptables processes rules from top to bottom, so specific ACCEPT rules must come before a broad DROP.

---

## 8. Allow DNS

For DNS queries:

```
sudo iptables -A INPUT -p udp --dport 53 -j  
ACCEPT
```

For TCP DNS traffic:

```
sudo iptables -A INPUT -p tcp --dport 53 -j  
ACCEPT
```

---

## 9. Allow established connections

A common rule is:

```
sudo iptables -A INPUT -m conntrack --ctstate  
ESTABLISHED,RELATED -j ACCEPT
```

This allows packets belonging to already-established connections.

---

## 10. Check the final rules

```
sudo iptables -L INPUT -n -v --line-numbers
```

Example:

```
1 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp  
dpt:22
```

```
2 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp  
dpt:80
```

```
3 DROP tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:21
```

```
4 DROP tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:23
```

### Quick lab example

**Allow:** SSH 22, HTTP 80

**Block:** FTP 21, Telnet 23

```
sudo iptables -A INPUT -p tcp --dport 22 -j  
ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 80 -j  
ACCEPT
```

```
sudo iptables -A INPUT -p tcp --dport 21 -j  
DROP
```

```
sudo iptables -A INPUT -p tcp --dport 23 -j  
DROP
```

Then verify with:

```
sudo iptables -L -n -v
```

You can test the result from your Kali lab machine with:

```
nmap -p 21,22,23,80 <TARGET-IP>
```

This gives you a practical **firewall** → **scan** → **verify** exercise.

- Demonstrate blocking a port scan attempt.

For your **Kali + Metasploitable 2 lab**, you can demonstrate how a firewall rule changes the result of an nmap scan.

### 1. Check the target's open ports

From Kali, first scan your **own Metasploitable 2 VM**:

```
nmap -sT -p 21,22,23,80,443  
<METASPLOITABLE-IP>
```

You may see results such as:

```
21/tcp  open  ftp  
22/tcp  open  ssh  
23/tcp  open  telnet  
80/tcp  open  http
```

The exact results depend on your VM configuration.

---

## **2. Block a port with iptables**

On the Metasploitable 2 VM, for example, block FTP:

```
sudo iptables -A INPUT -p tcp --dport 21 -j  
DROP
```

Check the rule:

```
sudo iptables -L INPUT -n -v --line-numbers
```

You should see a rule similar to:

```
DROP tcp -- 0.0.0.0/0 0.0.0.0/0 tcp dpt:21
```

---

### 3. Scan again from Kali

```
nmap -sT -p 21,22,23,80 <METASPLOITABLE-IP>
```

The previously accessible FTP port should now be blocked/filtered.

You might see:

| PORT | STATE | SERVICE |
|------|-------|---------|
|------|-------|---------|

|        |          |     |
|--------|----------|-----|
| 21/tcp | filtered | ftp |
|--------|----------|-----|

|        |      |     |
|--------|------|-----|
| 22/tcp | open | ssh |
|--------|------|-----|

|        |      |        |
|--------|------|--------|
| 23/tcp | open | telnet |
|--------|------|--------|

|        |      |      |
|--------|------|------|
| 80/tcp | open | http |
|--------|------|------|

filtered means Nmap cannot determine that the port is reachable because firewall filtering is interfering with the probes.

---

### 4. Demonstrate with Wireshark

Start Wireshark on the lab interface and use:

```
tcp.port == 21
```



Run the Nmap scan again.

You can observe TCP packets from the scanning machine and compare the behavior **before and after** the firewall rule.

---

## 5. Remove the rule after the demonstration

List rules with numbers:

```
sudo iptables -L INPUT -n --line-numbers
```

If the FTP DROP rule is number 3:

```
sudo iptables -D INPUT 3
```

Verify:

```
sudo iptables -L INPUT -n
```